

chat to estudando paradigmas, linguagem da vez é
prolog- to usando o terminal swish, me da um resumo da
parte de sintaxes, as coisas mais importantes q vou usar

Boa sorte, Caio. 🤗 Depois de Haskell e Lisp, agora chegou a vez do Prolog, que é bem diferente porque você não programa dizendo *como fazer*, mas sim descrevendo *fatos e regras*, e o Prolog descobre as respostas sozinho.

1. Estrutura básica

Tudo em Prolog termina com ponto (.)

Fato

```
prolog
```

Significa:

- Caio gosta de pizza
- Marcus gosta de hambúrguer

Consulta

No SWISH:

```
prolog
```

Resultado:

```
prolog
```

2. Variáveis

```
gosta(caio, pizza).
```

— — — — —

prolog

prolog

cabeca é verdadeira SE corpo for verdadeiro

```
homem(caio).
```

— — — — —

prolog

2/13

```
prolog
```

4. Várias condições (AND)

Usa vírgula.

```
pai(joao, maria).  
mae(ana, maria).
```

```
prolog
```

```
mae(M, X).
```

Significa:

```
X tem pais se existir um pai e uma mãe.
```

5. OU (OR)

Usa ponto e vírgula.

```
feliz(X) :-
```

```
prolog
```

Significa:

```
X é feliz se for rico OU famoso.
```

6. Comparações

Igual

prolog

==

Diferente

prolog

Exemplo:

prolog

Resultado:

prolog

Numéricas

Maior

prolog

Menor

prolog

Maior ou igual

prolog

Menor ou igual

```
prolog
```

Observe:

```
prolog
```

não é:

```
prolog
```

7. Atribuição matemática

Em Prolog não existe:

```
prolog
```

Isso gera:

```
prolog
```

(ele guarda a expressão)

Para calcular:

```
prolog
```

Resultado:

```
prolog
```

Essa palavra `is` aparece MUITO.

8. Listas

Lista simples

```
prolog
```

Cabeça e cauda

Muito importante.

```
prolog
```

Exemplo:

```
prolog
```

vira

```
prolog
```

Consulta:

```
prolog
```

Resultado:

```
prolog
```

```
T = [2,3]
```

9. Recursão

Equivalente ao loop.

Tamanho de lista

```
tamanho([],0).
```

```
prolog
```

```
IN 13 IN1 T 1.
```

10. Anonymous Variable

Quando não quer usar uma variável:

```
prolog
```

Exemplo:

```
pai(joao,maria).
```

```
prolog
```

```
PAI X Y
```

O nome do pai não importa.

11. Corte (Cut)

Símbolo:

```
prolog
```

Serve para impedir que o Prolog continue procurando respostas.

Exemplo:

```
maior(X,Y,X) :-  
    X >= Y,  
  
    prolog  
  
maior(_,_,_).
```

Muito usado em exercícios de faculdade.

12. Negação

prolog

Exemplo:

prolog

Lê-se:

```
não consegue provar que gosta de pizza
```

Estrutura que costuma cair em exercícios

Normalmente o monitor passa algo parecido com isso:

```
% fatos  
produto(teclado, 100).  
produto(mouse, 50).  
produto(monitor, 800).
```

prolog


```
produto(X, Preço),  
Preço > 200.
```

Consultas:

```
?- caro(monitor).  
true.
```

```
?- caro(mouse).  
false.
```

prolog

```
P = 100.
```

O que decorar para a prova

Se eu tivesse 10 minutos para estudar Prolog, decoraria:

```
: -          % regra  
  
,          % AND  
  
;          % OR  
  
is         % cálculo  
  
=          % igual  
  
\=         % diferente  
  
> < >= =<  % comparações  
  
[H|T]      % lista  
  
prolog  
  
!          % cut  
  
\+         % negação
```

E principalmente lembrar que:

Em Prolog você não faz um algoritmo passo a passo. Você descreve fatos e regras, e o interpretador encontra a solução através de busca lógica. Isso é a maior diferença em relação a Java, Python, Haskell e Lisp.



1. Sabendo que a Sucuri é alimentada pela Água e pela Terra, inclua esses fatos.
2. Crie uma regra especie_chave(Animal) que seja verdadeira SE o Animal se alimentar da Água OU da Terra E tiver um nicho associado a 'rios', 'pântano' ou 'profundezas'.
3. Query: Quem são as espécies chaves.

separa oq preciso realizar

Você precisa fazer 3 coisas:

1. Criar os fatos da Sucuri

Como o enunciado diz que a Sucuri se alimenta de **Água e Terra**:

prolog

2. Criar a regra especie_chave(Animal)

Ela será verdadeira se:

```
Animal se alimenta da água OU da terra
E
Animal tem nicho em rios, pântano ou profundezas
```

Em Prolog:

```
especie_chave(Animal) :-
    (alimenta(Animal, agua);
    alimenta(Animal, terra)),
```

prolog

```
nicho(Animal, pantano);  
nicho(Animal, profundezas)).
```

3. Fazer a query

Para descobrir quem são as espécies chave:

prolog

Ou seja: você adiciona os fatos da Sucuri, cria a regra e depois roda a consulta.



esse é mais simples, assim esta correto?

Manda o código ou um print dele 🤖

Pela imagem eu só consigo ver o enunciado. Cola aqui o que você escreveu em Prolog que eu verifico:

- se a sintaxe está correta;
- se a lógica bate com o que o professor pediu;
- se dá para simplificar;
- e qual query usar no SWISH.

Pode mandar o código inteiro que eu corrijo linha por linha.



```
alimenta(sucuri, agua).  
alimenta(sucuri, terra).
```

```
nicho(sucuri, rios).
```

```
especie_chave(Animal) :-  
    (alimenta(Animal, agua);  
    alimenta(Animal, terra)),
```

(nicho(Animal. rios):

Mostrar mais ▾

Sim, está correto ✓

A lógica ficou exatamente como o enunciado pediu.

A query seria:

```
prolog
```

Resultado esperado:

```
prolog
```

Só cuidado: se o professor usar "pântano" sem acento no código, mantenha como você fez:

```
prolog
```

Em Prolog é melhor evitar acento mesmo.



Exercício 2

1. Crie uma regra `especie_dominante(Animal)` que seja verdadeira SE o Animal tiver pelo menos dois nicho (domínios) associados no banco de dados E seu `local_comum` for as 'margens'.
2. Query: Quem é(são) a(s) espécie(s) dominante(s)? (Dica: Você precisará testar se `nicho(Animal, X)` e `nicho(Animal, Y)` com `X \= Y`).

travei em como ficaria a query aqui

A query fica simples assim:

prolog

```
?- especie_dominante(Animal).
```

O que você precisa montar antes é a **regra**:

prolog

```
especie_dominante(Animal) :-  
    nicho(Animal, X),  
    nicho(Animal, Y),  
    X \= Y,  
    local_comum(Animal, margens).
```

Ela lê assim:

Animal é dominante se tem um nicho X, tem outro nicho Y, X é diferente de Y, e o local comum dele é margens.

Exemplo da consulta:

prolog

```
?- especie_dominante(Animal).
```

Resultado esperado:

prolog

```
Animal = algum_animal.
```

